

Práctica Git: Ligar un repositorio Git local con GitHub

Objetivo

Al finalizar la práctica, el alumno será capaz de:

- Crear un repositorio local con Git.
- Crear un repositorio en GitHub.
- Conectar ambos repositorios.
- Subir su proyecto a GitHub.
- Realizar cambios y sincronizarlos con GitHub.

Conceptos Básicos.

¿Qué es Git?

Git es un sistema de control de versiones distribuido.

Su función es registrar los cambios que se realizan en los archivos de un proyecto a lo largo del tiempo.

Con Git puedes:

- Guardar diferentes versiones de un proyecto.
- Recuperar versiones anteriores.
- Saber quién hizo cada cambio.
- Trabajar sin conexión a Internet.
- Colaborar con otras personas.

Idea clave: Git está instalado en tu computadora.

Ejemplo

Supongamos que hoy escribes un programa:

```
System.out.println("Hola");
```

Mañana lo modificas:

```
System.out.println("Hola Mundo");
```

Con Git puedes regresar a la versión anterior cuando quieras.

¿Qué es GitHub?

GitHub es una plataforma en Internet que almacena repositorios Git.

Su función es permitir que varias personas compartan y colaboren en proyectos utilizando Git.

Con GitHub puedes:

- Respalda tus proyectos en la nube.
- Compartir código.
- Colaborar con otras personas.
- Descargar proyectos.
- Revisar cambios realizados por otros.

Idea clave: GitHub funciona sobre Git, pero no son lo mismo.

Analogía

Imaginen que están escribiendo una tesis.

Git

Es como guardar diferentes versiones del documento en tu computadora.

Tesis_v1.docx

Tesis_v2.docx

Tesis_v3.docx

Pero Git hace esto automáticamente y de forma organizada.

GitHub

Es como guardar esa tesis en Google Drive para que otras personas puedan verla o colaborar contigo.

Diferencias

Git	GitHub
Es un programa.	Es un sitio web.
Se instala en la computadora.	Funciona en Internet.
Controla versiones.	Aloja repositorios Git.
Puede usarse sin Internet.	Requiere Internet para sincronizar.
Guarda el historial de cambios.	Permite compartir ese historial.
Se usa mediante comandos (<code>git</code>).	Se usa desde el navegador o aplicaciones.

Ejemplo visual

Mi computadora

Proyecto Java

|



Git (historial)

|

git commit guarda versiones

|



git push

|



GitHub (Internet)

Respaldo y colaboración

¿Qué hace cada uno?

Git

- Guarda versiones.
- Recupera versiones.
- Compara cambios.
- Permite crear ramas.
- Fusiona cambios.

GitHub

- Almacena repositorios.
- Permite descargar proyectos.
- Permite colaborar.
- Maneja permisos.
- Muestra el historial desde la web.

¿Se necesita GitHub para usar Git?

No.

Puedes usar Git durante años sin crear una cuenta de GitHub.

Ejemplo:

Mi computadora

Proyecto

|



Git

Todo funciona localmente.

¿Se puede usar GitHub sin Git?

Prácticamente no.

GitHub fue creado para almacenar repositorios Git.

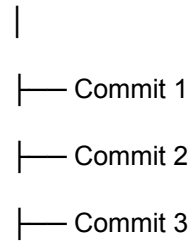
Ejemplo real

Supongamos que desarrollas una aplicación.

Solo con Git

Computadora

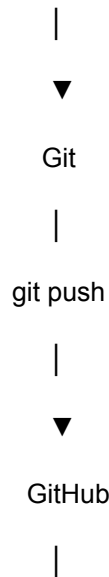
Proyecto



Todo queda únicamente en tu computadora.

Con Git y GitHub

Computadora



Otros desarrolladores

Ahora cualquier integrante del equipo puede descargar el proyecto.

Resumen para recordar

Git

Es la herramienta que lleva el historial de los cambios de un proyecto.

GitHub

Es la plataforma donde se almacenan y comparten proyectos que utilizan Git.

Frase que suele ayudar a los principiantes

**Git trabaja en tu computadora; GitHub trabaja en Internet.
Git guarda versiones; GitHub las comparte y sincroniza.**

Manual para enlazar Git con GitHub

Paso 1. Crear un proyecto

Supongamos que existe una carpeta llamada:

```
MiProyecto
|
|— src
|   |— Hola.java
|— README.md
```

Entrar a la carpeta:

```
cd MiProyecto
```

Paso 2. Inicializar Git

```
git init
```

Resultado:

Initialized empty Git repository in ...

Explicar:

Se crea una carpeta oculta llamada **.git**, donde Git almacenará todo el historial del proyecto.

Verificar:

```
git status
```

Paso 3. Agregar archivos

```
git add .
```

Explicar:

- **add** mueve los archivos al **Staging Area**.
- Todavía no forman parte del historial.

Verificar:

```
git status
```

Paso 4. Crear el primer commit

```
git commit -m "Primer commit"
```

Explicar:

Un **commit** es una fotografía del proyecto en un momento determinado.

Paso 5. Crear un repositorio en GitHub

Entrar a:

<https://github.com>

1. Iniciar sesión.
2. Presionar **New Repository**.
3. Escribir el nombre.

Ejemplo:

MiProyecto

Importante

No marcar:

-  Add README
-  Add .gitignore
-  Add License

Porque el proyecto ya existe localmente.

Paso 6. Copiar la URL

GitHub mostrará algo parecido a:

`https://github.com/usuario/MiProyecto.git`

Explicar:

Esta dirección será el repositorio remoto.

Paso 7. Conectar Git con GitHub

```
git remote add origin https://github.com/usuario/MiProyecto.git
```

Explicar:

- **remote** → repositorio remoto.
- **add** → agregar.
- **origin** → nombre del repositorio remoto (es una convención).
- La URL es la dirección del repositorio en GitHub.

Verificar:

```
git remote -v
```

Resultado esperado:

origin https://github.com/usuario/MiProyecto.git (fetch)
origin https://github.com/usuario/MiProyecto.git (push)

Paso 8. Subir el proyecto

git branch -M main

Explicar:

Renombra la rama principal a **main**.

Después:

git push -u origin main

Explicar:

- **push** → subir cambios.
- **origin** → repositorio remoto.
- **main** → rama principal.
- **-u** → recuerda esta configuración para futuros **push**.

Paso 9. Verificar

Actualizar GitHub.

Ya debe aparecer el proyecto.

Paso 10. Hacer un cambio

Modificar un archivo.

Después:

git status
git add .
git commit -m "Se agrega una nueva función"
git push

Explicar:

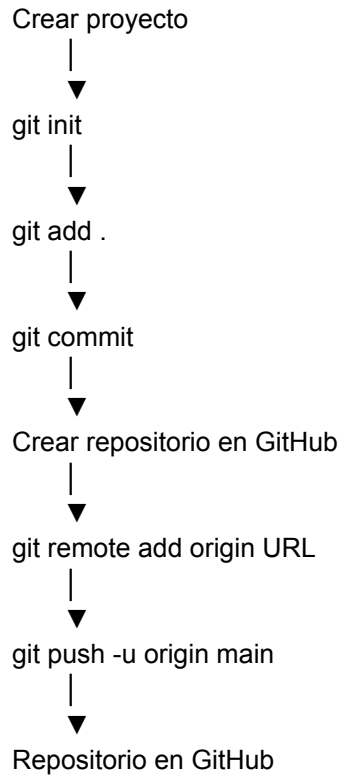
Como ya usamos

git push -u origin main

ahora basta con escribir:

`git push`

Flujo completo



Comandos que deben aprender

`git init`
`git status`
`git add .`
`git commit -m "mensaje"`
`git remote add origin URL`
`git remote -v`
`git branch -M main`
`git push -u origin main`
`git push`

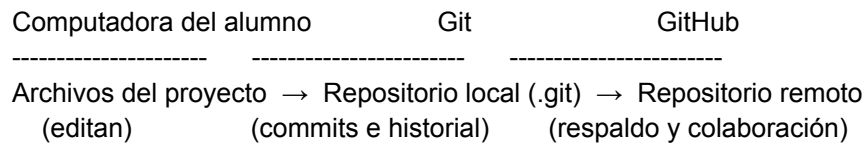
Preguntas que puedes hacer en clase

1. ¿Qué crea el comando `git init`?
2. ¿Qué diferencia existe entre `git add` y `git commit`?
3. ¿Qué es un repositorio remoto?
4. ¿Qué significa `origin`?
5. ¿Para qué sirve `git remote -v`?

6. ¿Qué hace `git push`?
7. ¿Por qué solo la primera vez se utiliza `git push -u origin main`?

Sugerencia didáctica

Como es su primer contacto con Git y GitHub, les recomiendo enseñar el proceso en **tres etapas**, para que comprendan qué ocurre en cada una:



Con este esquema visual, los alumnos entienden que:

- `git add` mueve archivos al área de preparación.
- `git commit` guarda una versión en el repositorio local.
- `git push` envía esas versiones al repositorio remoto en GitHub.

Esta separación de conceptos suele evitar muchas confusiones en las primeras prácticas.